Arithmetic Analysis of Polymorphic Programs
and the Verification of Typed Agent Interfaces

Rawle Prince*

October 2026

Abstract

We exploit the representation of data types as container functors to develop a decision procedure for a large class of polymorphic functions on lists. A container describes a data type by the *shapes* its values can take and, for each shape, the *positions* at which data are stored; a representation theorem guarantees that every polymorphic function between container types is given by a map on shapes together with a family of maps on positions. For lists, shapes are natural numbers and positions are finite types. We show that the behaviour of the dependently typed position maps can be captured by functions on the natural numbers, and that when both shape and position maps are definable in Presburger arithmetic, equality of polymorphic functions reduces to the validity of universally quantified formulae of Presburger arithmetic, which is decidable. The procedure is implemented as a Coq tactic that discharges the standard identities between list functions (reversal, appending, taking, dropping, rotation, ...) without induction, each in a fraction of a second.

We then develop a second reading of the same mathematics. A container is also the type of an *interface*: the moves an agent can emit and, for each move, the responses it accepts. Container morphisms are then *adapters* between interfaces (dependent lenses), and the composition of adapters is the composition of container morphisms. Interfaces whose shape is a count — batches of tool calls, context windows, ranked candidate lists — are list-shaped, and the adapters between them are exactly the list morphisms that our procedure decides. We show that arithmetic equality of adapters implies behavioural equality of every agent wrapped by them, that losslessness (cartesianness) of an adapter is decidable, and we verify, and in two cases refute, invariants of sliding context windows and of fan-out/fan-in routing. The accompanying Coq development is new, self-contained and axiom-free.

1 Introduction

Techniques for reasoning about polymorphic functions have applications that range from specification and verification, through model checking and testing, to the certification of mobile code. This paper presents a way of reasoning about polymorphic functions that is based on the representation of data types as *containers* [1, 2, 3]. A container describes a concrete data type by two pieces of information: the *shapes* that values of the type can take, and, for each shape, the set of *positions* at which data are stored. A list, for instance, has as shape its length n and as positions the n slots $0, \ldots, n-1$; a binary tree has as shape a tree with no data in it, and as positions the nodes of that tree. A value of the type is then a shape together with an assignment of a datum to each position.

Earlier presentations of container theory used the language of category theory [2, 3] and investigated its applications to generic programming [7, 33]. Our interest comes from automated

*Tucan Analytics Ltd, London, United Kingdom. rawle.prince@tucan-analytics.com. The arithmetic part of this work originates in the author's doctoral research, supported by a departmentally funded studentship in the School of Computer Science, University of Nottingham (2006–2010).

reasoning, and is motivated by the ability of containers to simplify reasoning tasks [4, 39]. The basis of everything that follows is the *representation theorem* for containers: every polymorphic function (natural transformation) between container types is given by a map on shapes and, contravariantly, a family of maps on positions. For instance, a polymorphic function $F : \mathit{List}\, X \to \mathit{List}\, X$ is represented by a pair

$$\big(\, u : \mathbb{N} \to \mathbb{N}, \quad f : \textstyle\prod_{n:\mathbb{N}} \mathit{Fin}\,(u\,n) \to \mathit{Fin}\,n \,\big),$$

where u computes the length of the output from the length of the input, and, for each input length n, the *reindexing map* f_n says from which position of the input each position of the output is fetched. Here $\mathit{Fin}\,n$ is the type with exactly n elements, which we think of as $\{0, \ldots, n-1\}$. Reversal, for example, keeps the length ($u\,n = n$) and fetches output position i from input position $n - i - 1$.

Many functions on lists are, in this representation, nothing but arithmetic on lengths and indices. This was the insight behind Bundy and Richardson's *ellipsis* technique [14], in which lists are written as $x_1, \ldots, x_n$ and functions are specified by what they do to indices, and it was developed further, with containers as the formal underpinning, in [39]. Those developments were carried out in a dependently typed setting, where indices live in the finite types $\mathit{Fin}\,n$. Dependent types are what make the representation *correct by construction* — an index into a list of length n cannot exceed $n-1$ — but they are also what prevents the direct use of decision procedures, which operate on natural numbers or integers, not on families of finite types.

1.1 Contribution: arithmetic verification

The first contribution of this paper is to show that, for lists, nothing is lost by abandoning dependent types at the point where decision procedures are needed. The behaviour of a reindexing map $f : \prod_n \mathit{Fin}(u\,n) \to \mathit{Fin}\,n$ is captured by a function $g : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ on natural numbers together with a side condition (the index is in range), and equality of two polymorphic functions becomes a statement of the form

$$\forall n.\; u\,n = v\,n \;\wedge\; \forall n\,i.\; i < u\,n \to g_1\,n\,i = g_2\,n\,i.$$

When u, v, g_1 and g_2 are definable in Presburger arithmetic — the first-order theory of the natural numbers with addition, which is decidable [36] — so is this statement, and because it has a universal prefix and a quantifier-free matrix it falls into the fragment handled efficiently by the Omega test [40], by Coq's `lia` tactic and by SMT solvers. We identify a class of *Presburger-definable morphisms*, closed under composition, that covers the familiar functions on lists (§4), and we implement a Coq tactic that proves the identities of Figure 4 automatically.

1.2 Contribution: typed agent interfaces

The second contribution was prompted by an observation of Ghani [20]. An *AI agent* responds to prompts: it has a type of possible outputs (moves, tool calls, messages) and, for each output, a type of inputs it is prepared to receive next. That is a container. The whole theory of containers — and its continuation as the theory of polynomial functors by Spivak and Niu [41, 34] — therefore applies to the *interfaces* of agents. In particular, a container morphism $(S \triangleleft P) \Rightarrow (S' \triangleleft P')$ is a protocol *adapter*: it translates the moves of one interface into those of another (covariantly) and routes responses back (contravariantly). In the terminology of [34] it is a *dependent lens*. Composition of adapters is composition of container morphisms, and wrapping an agent by an adapter is the extension of the container morphism applied to the agent's one-step behaviour.

Most of today's agentic systems are untyped: an LLM has, as Ghani puts it, type "string to string", and orchestration frameworks pass around lists of messages and JSON objects whose

shapes are checked, if at all, at run time. A *typed* agentic system, of the kind being built at Kodamai [25, 20], assigns container types to interfaces and composes agents along typed adapters. Once that is done, a familiar dividend of types becomes available: to check that two ways of wiring a system agree, or that an adapter preserves information, one checks the types and the adapters, not the agents.

Here is where the first contribution meets the second. A great many agent interfaces are *list-shaped*: an orchestrator that fans out n tool calls and fans in n results; a conversation whose context is a window over a history of length n; a retriever that returns k ranked documents. Adapters between list-shaped interfaces are list morphisms — truncate, drop the oldest, reverse the ranking, split and merge a batch — and they are exactly the Presburger-definable morphisms of §4. Consequently, (i) equality of two adapter pipelines is decidable; (ii) losslessness of an adapter (no response dropped or duplicated) is decidable; and (iii) by functoriality, an equation between adapters transfers to every agent wrapped by them. We prove these facts in Coq (§5) and verify, in well under a second, invariants of sliding context windows and of fan-out/fan-in routing; just as usefully, the procedure *refutes* two plausible-looking adapter laws, exhibiting the counterexamples.

1.3 Relation to earlier work

The arithmetic core (§2–§4) develops ideas first set out in the author's thesis [38] and, in dependently typed form, in [39]. Relative to those, the decidability theorem is given a proof (Theorem 4.7), correctness of composition is stated and proved (Theorem 4.11), and the accompanying Coq development has been rewritten: it is self-contained, axiom-free, uses `lia` in place of the deprecated `omega`, proves each identity of Figure 4 in a fraction of a second, and connects the container-style definitions with ordinary recursive definitions through *transfer lemmas*, from which standard theorems about lists follow without induction (§4.6).[1] The interface reading (§3 and §5) is new.

1.4 Notation and conventions

We use two notations side by side, and we say which is which. Mathematical statements and definitions are written in a type-theoretic notation in the style of Agda and Epigram [32]: $\sum_{s:S} P\,s$ (or $\sum s : S.\, P\,s$) is the type of dependent pairs (s, p) with $s : S$ and $p : P\,s$; $\prod_{s:S} P\,s$ is the type of dependent functions f with $f\,s : P\,s$ for every s; and a definition by pattern matching lists one clause per constructor, with the *absurd clause* "!" marking a case that cannot arise (written `()` in Agda). Code from the accompanying development is shown in Coq syntax in typewriter font. In Coq, $\sum$ is `sigT` or `{x : A & B x}` and $\prod$ is `forall`. We write $S\,n$ for the successor of n in pattern-matching clauses and $n + 1$ in arithmetic; the two are identical. Cut-off subtraction on $\mathbb{N}$, written $n \mathbin{\dot{-}} m$ and equal to 0 when $m \geq n$, is the subtraction of Coq's `nat`. Lists are indexed from 0 throughout.

Accompanying development. All definitions and results marked [Coq] are formalised in the development (directory `theories/`) at `https://github.com/tucananalytics/AAPP` of which §4.6 gives an overview.

2 Containers

We begin with an informal motivation, then give the definitions we need. Readers familiar with containers may skip to §2.4; readers wishing to see the full theory should consult [3].

[1] The earlier Coq sources are kept in the `legacy` folder of the repository given in §1.4.

2.1 Motivation: unique representations of lists

Bundy and Richardson [14] represent a list of elements of type τ by a length and a function that produces its elements:

$$\begin{aligned} \square &: \mathbb{N} \to (\mathbb{N} \to \tau) \to \mathit{List}\,\tau \\ \square\,(0, f) &= [\,] \\ \square\,(S\,n, f) &= f\,1 :: \square\,(n, f \circ S). \end{aligned}$$

Thus $\square(n, f)$ is the list $[f\,1, f\,2, \ldots, f\,n]$, and functions on lists can often be written as arithmetic on n and on the arguments of f. The difficulty with this representation is that $\square$ is not injective: the lists $\square(2, f)$ and $\square(2, f')$ are equal as soon as f and f' agree on 1 and 2, whatever they do elsewhere, so a list has many representations and f carries information that is never observed. The cure is to restrict the domain of f to the positions that exist, that is, to make its domain depend on n. Using the finite types

```
Inductive Fin : nat -> Set :=
  | fz : forall n, Fin (S n)
  | fs : forall n, Fin n -> Fin (S n).
```

(so $\mathit{Fin}\,0$ is empty, and $\mathit{Fin}(S\,n)$ has the elements fz and $\mathit{fs}\,i$ for $i : \mathit{Fin}\,n$ — we write fs in italics to avoid reading it as an application $f\,s$), the type

$$\Phi\,X \;=\; \textstyle\sum_{n:\mathbb{N}} (\mathit{Fin}\,n \to X)$$

represents lists uniquely: a pair (n, f) with $f : \mathit{Fin}\,n \to X$ determines the list $[f\,0, \ldots, f\,(n-1)]$ and is determined by it. Φ is a functor, with $\Phi\,g\,(n, f) = (n, g \circ f)$. It turns out that Φ is precisely the representation of lists that container theory produces.

2.2 Containers and their extensions

Definition 2.1 (Container)**.** A *container* $(S \triangleleft P)$ consists of a type S of *shapes* and, for each shape $s : S$, a type $P\,s$ of *positions*.

We sometimes write a container in pointwise form, $(s : S \triangleleft P\,s)$, to make the shape variable explicit. The *list container* is $(n : \mathbb{N} \triangleleft \mathit{Fin}\,n)$: a shape is a length and the positions of a list of length n are its n slots. The container of binary trees has as shapes the trees with no data and as positions, for each such tree, its nodes.

Definition 2.2 (Extension)**.** The *extension* of a container $(S \triangleleft P)$ is the functor $T_{(S \triangleleft P)} : \mathbf{Set} \to \mathbf{Set}$ defined by

$$T_{(S \triangleleft P)}\,X \;=\; \textstyle\sum_{s:S} (P\,s \to X),$$

with action on functions $T_{(S \triangleleft P)}\,g\,(s, h) = (s, g \circ h)$.

A value of $T_{(S \triangleleft P)}\,X$ is a shape together with an assignment of an element of X to every position of that shape. For the list container, $T_{(\mathbb{N} \triangleleft \mathit{Fin})}\,X = \sum_n (\mathit{Fin}\,n \to X) = \Phi\,X \cong \mathit{List}\,X$.

Remark 2.3 (Calculating shapes). The shapes of an algebraic data type can be read off by setting the payload type to the unit type: the shape type of $\mathit{List}\,X$ is $\mathit{List}\,\mathbf{1} \cong \mathbb{N}$ and that of $\mathit{Btree}\,X$ is $\mathit{Btree}\,\mathbf{1}$. Containers are closed under the constructions of strictly positive types, given by the grammar

$$\tau \;::=\; X \;\mid\; \mathbf{0} \;\mid\; \mathbf{1} \;\mid\; \tau + \tau \;\mid\; \tau \times \tau \;\mid\; K \to \tau \;\mid\; \mu X.\,\tau,$$

in which X ranges over type variables, $\mathbf{0}$ and $\mathbf{1}$ are the empty and unit types, $+$ and $\times$ are sum and product, $K \to \tau$ is exponentiation by a constant type K (one with no free variables), and $\mu X.\,\tau$ forms the least fixed point, that is, the inductive type whose constructors are described by τ with X standing for recursive occurrences. For example, $\mathbb{N} = \mu X.\,\mathbf{1} + X$, $\mathit{List}\,A = \mu X.\,\mathbf{1} + A \times X$ and $\mathit{Btree}\,A = \mu X.\,A + X \times X$. See [3] for the construction of the container corresponding to each clause.

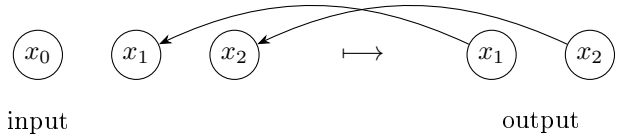


Figure 1: The reindexing map of *tail* sends each output position back to the input position it is fetched from. Input position 0 has no preimage: its datum is discarded.

2.3 Container morphisms

A polymorphic function between container types, say $rev : List\, X \to List\, X$, is determined by what it does to shapes and positions: *rev* keeps the length, and the datum at output position i is the datum at input position $n - i - 1$. Note the direction of the second map: it goes from *output* positions to *input* positions. This is deliberate. A polymorphic function cannot invent data, so every output position must be filled from some input position — hence a function from output positions to input positions. The opposite direction would not work in general: *tail* discards input position 0, which has nowhere to go (Figure 1), and a function that duplicates its input sends one input position to two output positions, which is not a function at all.

Definition 2.4 (Container morphism)**.** A *container morphism* $(u, f) : (S \triangleleft P) \Rightarrow (S' \triangleleft P')$ consists of

$$u : S \to S' \qquad \text{and} \qquad f : \textstyle\prod_{s:S} P'\,(u\,s) \to P\,s.$$

We call u the *shape map* and f the *position* or *reindexing map*.

The shape map is covariant (it follows the direction of the function) and the position map is contravariant, for the reason just given. Each container morphism determines a polymorphic function, its extension:

$$mmap\,(u, f) : T_{(S \triangleleft P)}\, X \to T_{(S' \triangleleft P')}\, X, \qquad mmap\,(u, f)\,(s, g) = (u\,s,\ g \circ f_s). \tag{1}$$

In words: apply u to the shape, and fill output position p with the datum $g\,(f_s\,p)$ found at the input position $f_s\,p$.

Example 2.5 (Reverse)**.** $crev = (id, rv) : (\mathbb{N} \triangleleft Fin) \Rightarrow (\mathbb{N} \triangleleft Fin)$, where $rv_n\,i$ is the element of $Fin\,n$ with value $n - i - 1$. In the dependently typed style rv is defined by recursion on the index, using the functions $top_n : Fin(S\,n)$ (the largest element, of value n) and $emb_n : Fin\,n \to Fin(S\,n)$ (the value-preserving embedding):

$$\begin{array}{ccc} rv_{S\,n}\,fz = top_n & top_0 = fz & emb\,fz = fz \\ rv_{S\,n}\,(fs\,i) = emb\,(rv_n\,i) & top_{S\,n} = fs\,top_n & emb\,(fs\,i) = fs\,(emb\,i). \end{array}$$

Every element of $Fin(S\,n)$ is either top_n or $emb\,i$ for a unique i; this is the *view* [31, 32] used in the Coq code of §4.6.

Example 2.6 (Tail)**.** $ctail = (u, f)$ with $u\,0 = 0$, $u\,(S\,n) = n$, $f_0 = !$ (no output positions when the input is empty) and $f_{S\,n}\,i = fs\,i$ (output position i is fetched from input position $i + 1$).

Definition 2.7 (Cartesian morphism)**.** A container morphism (u, f) is *cartesian* if every $f_s : P'(u\,s) \to P\,s$ is a bijection.

A cartesian morphism rearranges data without loss or duplication; *crev* is cartesian, *ctail* is not. Note that cartesianness constrains the *position* maps only; the shape map u may well fail to be injective (§7 returns to this point).

Definition 2.8 (Composition)**.** Given $(u_1, f_1) : (S \lhd P) \Rightarrow (S' \lhd P')$ and $(u_2, f_2) : (S' \lhd P') \Rightarrow (S'' \lhd P'')$, their composite $(u_2, f_2) \circ (u_1, f_1) : (S \lhd P) \Rightarrow (S'' \lhd P'')$ is $(u_2 \circ u_1,\ \lambda s\, p.\ f_1\, s\, (f_2\, (u_1\, s)\, p))$.

Shapes are mapped forwards through u_1 then u_2; a position of the final output is routed backwards through f_2 then f_1. Containers and container morphisms form a category **Cont** [3], and *mmap* respects composition and identities ([Coq] `mmap_comp`, `mmap_id`).

2.4 The representation theorem and equality

Theorem 2.9 (Representation [2, 3])**.** *The map mmap is a bijection between container morphisms* $(S \lhd P) \Rightarrow (S' \lhd P')$ *and natural transformations* $T_{(S \lhd P)} \to T_{(S' \lhd P')}$.

Categorically the theorem is an instance of the Yoneda lemma, and it says that $T : \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ is a full and faithful embedding. For us it says two things. First, *every* polymorphic function between lists is of the form $mmap\,(u, f)$, so nothing is lost by working with shape and position maps. Second, two container morphisms with the same extension are equal, so an equation between polymorphic functions can be proved by proving an equation between container morphisms. (The core theory, including this theorem, was formalised in Coq in [38, Ch. 4].)

Stating the equality requires some care. Two container morphisms $(u, f), (u', f') : (S \lhd P) \Rightarrow (S' \lhd P')$ have components of types

$$u, u' : S \to S', \qquad f : \textstyle\prod_s P'(u\, s) \to P\, s, \qquad f' : \textstyle\prod_s P'(u'\, s) \to P\, s.$$

Two issues arise. *Heterogeneity*: f and f' have different types unless one already knows $u = u'$, so the ordinary (homogeneous) equality of type theory cannot even be used to state $f = f'$. *Extensionality*: we wish to regard two functions as equal when they agree on all arguments, not only when they have the same definition.

The standard remedy for the first issue is McBride's heterogeneous equality [30, 31], written $a \overset{jm}{=} b$ and also called "John Major equality". Its rules are:

$$\frac{a : A \qquad b : B}{a \overset{jm}{=} b : \mathit{Prop}} \qquad \frac{a : A}{\mathit{refl} : a \overset{jm}{=} a} \qquad \frac{a : A \qquad \Phi : \forall a' : A.\, a \overset{jm}{=} a' \to \mathit{Type} \qquad \phi : \Phi\, a\, \mathit{refl}}{\forall a' : A.\, \forall q : a \overset{jm}{=} a'.\, \Phi\, a'\, q}$$

These should be read as follows. The formation rule says that $a \overset{jm}{=} b$ is a proposition for *any* a and b, even of different types — so it can be *stated*. The introduction rule says that the only way to *prove* it is reflexivity, which requires the same object on both sides, and hence the same type. The elimination rule says that a proof of $a \overset{jm}{=} a'$ may be *used* like a proof of ordinary equality, but only when a and a' have the same type A. Thus $\overset{jm}{=}$ lets us write down equations between objects of distinct types, while anything we can conclude from such an equation requires the types to coincide. The second issue is handled by comparing functions pointwise.

Definition 2.10 (Equality of container morphisms)**.** For $(u, f), (u', f') : (S \lhd P) \Rightarrow (S' \lhd P')$ define

$$\frac{\forall s : S.\, u\, s = u'\, s \qquad \forall s : S.\, \forall p : P'(u\, s).\, \forall p' : P'(u'\, s).\, p \overset{jm}{=} p' \to f\, s\, p \overset{jm}{=} f'\, s\, p'}{(u, f) \overset{eqmor}{=} (u', f')} \tag{2}$$

The first premise says that the shape maps agree. The second says that the position maps agree: for every shape s and every pair of "the same" output position seen through u and through u' (which is what $p \overset{jm}{=} p'$ expresses, given the first premise), the two input positions computed by f and f' coincide. Since $f\, s\, p$ and $f'\, s\, p'$ both have type $P\, s$, the conclusion of the second premise is a homogeneous equality in disguise.

3 Containers as Agent Interfaces

This section explains how the material of §2 is read as a theory of typed interfaces for AI agents. Everything here is elementary; the point is to fix definitions precisely enough that the verification results of §5 can be stated. The development owes much to Spivak and Niu's account of polynomial functors as a "mathematical theory of interaction" [41, 34]; polynomial functors and containers are the same objects, and a container morphism is what [34] calls a dependent lens.

3.1 Interfaces

Definition 3.1 (Interface)**.** An *interface* is a container $I = (S \triangleleft P)$, read as follows: S is the type of *moves* (outputs) an agent with interface I may emit, and for each move s, $P\,s$ is the type of *responses* (inputs) the agent accepts after emitting s.

In the notation of [34] the interface is the polynomial $\sum_{s:S} y^{P\,s}$. Some examples:

- An untyped language model has interface $(\mathit{String} \triangleleft \lambda_.\, \mathit{String})$: it emits a string and accepts any string in return.
- A tool-calling agent has interface $(\mathit{Req} \triangleleft \mathit{Res})$: it emits a request $r : \mathit{Req}$ (a tool name with typed arguments) and accepts a result of the type $\mathit{Res}\,r$ *determined by that request.* The dependency is the whole point: the response type depends on the move. This is what structured-output and tool-use schemas [35, 29] enforce at the boundary of an LLM.
- A *batch* interface is the list container $(n : \mathbb{N} \triangleleft \mathit{Fin}\,n)$: the agent emits a batch of n items and accepts a response *per item.* More generally a batch of typed requests is the container $(\mathbf{r} : \mathit{Req}^m \triangleleft \mathit{Res}\,r_1 + \ldots + \mathit{Res}\,r_m)$, whose shapes are tuples of requests and whose positions are the result slots, each typed by its request.
- A *context window* over a history is list-shaped too: the shape is the length of the history the agent exposes, and positions are the turns within it.
- A *plan* is tree-shaped: a planner that decomposes a task into sub-tasks, recursively, emits a tree whose leaves are the atomic sub-tasks, and expects a result for each leaf. Its interface is the container of trees (§3.4).
- An *aggregator* that receives the batches of k sub-agents and presents them as one batch has an interface with k counts for its shape; adapters out of it are the arity-k morphisms of §4, of which $+\!\!+$ is the simplest.

Interfaces compose. If I and J are interfaces then so is their product (emit a move of each, accept a response to each), their sum (emit a move of one or the other), and their composite $I \circ J$ (emit an I-move whose positions are filled with J-moves); containers are closed under all of these [3], and so are polynomial functors [34].

The extension $T_I\,X$ (Definition 2.2) has a natural reading as well: an element (s, g) of $T_I\,X$ is a move s together with a *continuation* $g : P\,s \to X$ that says what to do with each possible response. For the batch interface, (n, g) is a batch of n slots each filled with an X — a list. Both readings are used below.

3.2 Adapters

Definition 3.2 (Adapter)**.** An *adapter* from interface $I = (S \triangleleft P)$ to interface $J = (S' \triangleleft P')$ is a container morphism $(u, f) : I \Rightarrow J$.

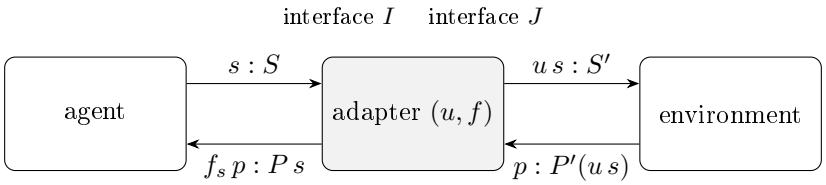


Figure 2: An adapter $(u, f) : I \Rightarrow J$ translates moves forwards and responses backwards. Seen from the environment, the adapted agent has interface J.

Unfolding Definition 2.4: $u : S \to S'$ translates each move of I into a move of J, and $f_s : P'(u\, s) \to P\, s$ translates each response that J may give to the move $u\, s$ back into a response that the I-agent understands (Figure 2). The contravariance of f, which was forced on us in §2.3 by the nature of polymorphic functions, is here forced by the nature of protocols: an adapter that sits between an agent and its environment must translate outgoing traffic forwards and incoming traffic backwards. No "lens laws" are imposed; the adapters of interest are rarely well-behaved in the sense of bidirectional programming, and none is needed.

Example 3.3. (i) *Reversal of a ranking.* A retriever exposes k candidates best-first; a consumer expects worst-first. The adapter is *crev* (Example 2.5): the batch is re-indexed, and a response to candidate i in the consumer's numbering is routed to candidate $k - i - 1$ in the retriever's. (ii) *Context window.* An agent exposes a history of length n to a model that accepts at most k turns. The adapter $last_k$ has shape map $u\, n = \min(n, k)$ and position map $f_n\, i = i + (n \mathbin{\dot{-}} k)$: window position i is history position $i + (n - k)$ when $n > k$, and i otherwise. (iii) *Fan-out and fan-in.* An orchestrator splits a batch of n requests at index m, dispatches the two parts, and reassembles the results. Splitting is the pair of adapters $take_m$, $drop_m$; reassembly is $+\!\!+$.

3.3 Agents, wrapping, and compositional verification

Definition 3.4 (Agent)**.** An *agent* with interface I and state type X is a coalgebra of the extension functor: a function

$$a : X \to T_I\, X, \qquad a\, x = (\mathit{readout}\, x,\ \mathit{update}\, x),$$

giving, for each state, the move the agent emits and, for each possible response, the next state.

This is the familiar notion of a Moore machine with a dependently typed input alphabet, and it is precisely a lens $X\, y^X \to I$ in the sense of [34]. Running an agent against an environment strategy $e : \prod_s P\, s$ (which picks a response to every move) produces the sequence of moves $\mathit{readout}\, x_0, \mathit{readout}\, x_1, \ldots$ with $x_{j+1} = \mathit{update}\, x_j\, (e\, (\mathit{readout}\, x_j))$ ([Coq] `run`).

Definition 3.5 (Wrapping)**.** Given an adapter $m : I \Rightarrow J$ and an agent $a : X \to T_I\, X$, the *wrapped agent* $\mathit{wrap}\, m\, a : X \to T_J\, X$ is $\mathit{mmap}\, m \circ a$.

That is, the wrapped agent emits the move $u\, (\mathit{readout}\, x)$ and, on response p, moves to the state $\mathit{update}\, x\, (f\, (\mathit{readout}\, x)\, p)$. Three properties follow immediately from the corresponding properties of *mmap*; in the development they are the lemmas `wrap_id`, `wrap_comp` and `wrap_ext_eq`.

Proposition 3.6 (Compositionality of wrapping)**.**

(i) $\mathit{wrap}\, \mathit{id}\, a = a$.

(ii) $\mathit{wrap}\, (m_2 \circ m_1)\, a = \mathit{wrap}\, m_2\, (\mathit{wrap}\, m_1\, a)$.

(iii) If $\mathit{mmap}\, m$ and $\mathit{mmap}\, m'$ agree on the one-step behaviour of a, then $\mathit{wrap}\, m\, a$ and $\mathit{wrap}\, m'\, a$ produce the same run against every environment, for every number of steps.

Item (ii) is the "scalability" dividend of types described in [20]: a pipeline of adapters is itself an adapter, and it is computed on the adapters alone. Item (iii) is the "trustworthiness" dividend: to establish that two wirings of a system have the same behaviour, it suffices to establish an equation between adapters, which by Theorem 2.9 is an equation between shape and position maps. Nothing need be known about the agent itself — in particular nothing about the language model that may sit inside it. We are, with Ghani [20], verifying the plumbing, not the engine.

Example 3.7 (A conversational agent and its window)**.** Let the state of an agent be its history $h : \mathit{List}\ \mathit{Turn}$, and let its interface be the batch interface $(n : \mathbb{N} \triangleleft \mathit{Fin}\, n)$ with positions read as turns. The agent a exposes its whole history ($\mathit{readout}\, h = |h|$, with the i-th position carrying the i-th turn in the extension) and, on receiving a response at position i — say a correction to turn i — updates that turn and appends the exchange. The model that serves the agent accepts at most k turns. Rather than change the agent, we wrap it: $\mathit{wrap}\, \mathit{last}_k\, a$ exposes $\min(|h|, k)$ turns, the most recent ones, and routes a response at window position i to history position $i + (|h| \dot{-} k)$ — the correct turn. The identities of §5.3 then say, for instance, that an implementation which stores only the last k turns and one which stores everything and windows on demand present the same interface and the same behaviour to the model, whatever the agent does with corrections.

3.4 Tree-shaped interfaces: plans and flattening

Not every interface is list-shaped, and the most important non-list example in agentic systems is the *plan*. A planner decomposes a task into sub-tasks and those into sub-sub-tasks; the result is a tree whose leaves are the atomic actions to be carried out, and the planner expects one result per leaf. As a container, the interface of such a planner is

$$\mathit{Tree} \;=\; (s : \mathit{TreeS} \triangleleft \mathit{leaves}\, s),$$

where TreeS is the type of tree skeletons (binary trees with no data, in the simplest case) and $\mathit{leaves}\, s$ is the type of leaf positions of s. Hierarchical delegation of this kind is studied in depth by Libkind and Spivak [28], with polynomial functors as task–solution interfaces and lenses as delegations; here we need only one adapter out of it.

The environment that executes the atomic actions — a tool server, a pool of workers — typically understands batches, not trees. The adapter that mediates is *flattening*:

$$\mathit{flatten} = (\mathit{Sum}, \mathit{flat}) : \mathit{Tree} \Rightarrow (\mathbb{N} \triangleleft \mathit{Fin}), \qquad \mathit{Sum}\, \mathit{leaf} = 1, \quad \mathit{Sum}\, (\mathit{node}\, l\, r) = \mathit{Sum}\, l + \mathit{Sum}\, r,$$

whose shape map counts the leaves and whose position map $\mathit{flat}_s : \mathit{Fin}(\mathit{Sum}\, s) \to \mathit{leaves}\, s$ sends the i-th slot of the batch to the i-th leaf in left-to-right order (Figure 3): if $i < \mathit{Sum}\, l$ the slot belongs to the left subtree, otherwise slot $i - \mathit{Sum}\, l$ of the right subtree. This is the container morphism `cflatten` of [38, Listing 21], and it has a property that matters for agents:

Proposition 3.8. *[Coq: `flatten`, `flatten_cartesian`, `Sum_not_injective`] flatten is cartesian: for every skeleton s, flat_s is a bijection, with inverse the left-to-right index of a leaf. Its shape map Sum is not injective.*

Read as an adapter, cartesianness says that wrapping a planner by *flatten* is *lossless*: the environment sees a flat batch of $\mathit{Sum}\, s$ actions, every result it returns is routed back to exactly one leaf, no leaf is left without a result and no result is delivered twice. Non-injectivity of the shape map says that the environment learns nothing about the *structure* of the plan — two plans with the same number of leaves present the same batch — which is as it should be: the plan

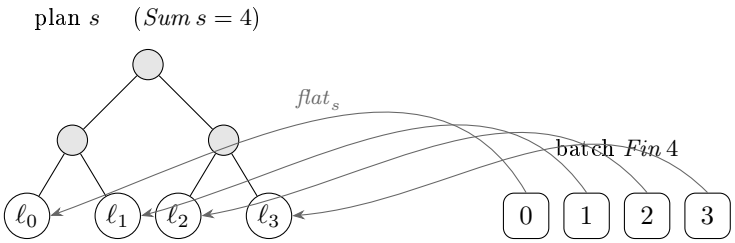


Figure 3: Flattening as an adapter from a tree-shaped interface to the batch interface. The position map sends batch slot i to the i-th leaf; it is a bijection, so no result is lost or duplicated.

is the planner's business. Cartesianness constrains positions, not shapes, and the distinction is exactly the one between data (which an adapter may not lose) and structure (which it may hide).

Flattening also connects tree-shaped interfaces to the decision procedure. By Proposition 3.6(ii), a list adapter m applied after flattening is the single adapter $m \circ \mathit{flatten}$ out of the tree interface, so a planner can be windowed, split across workers, or re-ordered using the adapters of §4, and the identities between those adapters — decided by `containers` — hold of the planner too. In the other direction, a tree adapter $t : \mathit{Tree} \Rightarrow \mathit{Tree}$ (re-balancing a plan, say) can be compared with a list adapter m through the equation $\mathit{flatten} \circ t = m \circ \mathit{flatten}$, which asks whether re-arranging the plan has the same effect on the batch as m. Since *flatten* is cartesian, an m satisfying the equation is determined by t wherever one exists, and the equation is then decidable whenever that m is Presburger. Section 7 returns to this.

3.5 Borders and list-shaped interfaces

An LLM produces text. Two routes lead from text to a typed interface [20]: change the model so that it produces structured output natively, or keep the model and parse its output against a schema at the boundary — "passport control". The second route is what structured-output APIs [35] and tool protocols [29] implement today: the schema is a type, the parser is a checker, and a parse failure is a type error caught before it propagates. Our concern in this paper is not the parser but what happens after it, once values of container types are flowing between components. Nor do we claim anything about the probabilistic internals of the model; the agent of Definition 3.4 is an arbitrary function of its state.

Within that scope, the observation on which §5 rests is simple: whenever the shape of an interface is a count — a batch size, a window length, a number of candidates — the interface is list-shaped, the adapters between such interfaces are list morphisms, and list morphisms are the subject of §4.

4 Arithmetic Verification

We now turn to deciding equality between list morphisms. We first show what a proof looks like in the dependently typed setting (§4.1) and why decision procedures do not apply directly; then we derive a non-dependent representation (§4.2) and identify a decidable class (§4.3).

4.1 The dependently typed setting

Consider proving that reversal is an involution. In container terms the statement is $(\mathit{crev} \circ \mathit{crev}) \overset{\mathit{eqmor}}{=} \mathit{id}$, and unpacking Definitions 2.8 and 2.10 leaves two obligations:

1. $\forall n : \mathbb{N}.\ n = n$, for the shapes, and
2. $\forall n : \mathbb{N}.\, \forall i : \mathit{Fin}\, n.\ \mathit{rv}_n\,(\mathit{rv}_n\, i) = i$, for the positions.

The first is trivial and the second is a lemma about *Fin* proved by induction on i. Appending is similar but needs more machinery. Its container morphism is

$$\mathit{cappn} : ((n, m) : \mathbb{N} \times \mathbb{N} \triangleleft \mathit{Fin}\, n + \mathit{Fin}\, m) \Rightarrow (\mathbb{N} \triangleleft \mathit{Fin}), \qquad u\,(n, m) = n + m,$$

with the reindexing map $f_{(n,m)} : \mathit{Fin}(n + m) \to \mathit{Fin}\, n + \mathit{Fin}\, m$ reflecting the sum structure: output positions below n come from the first list and the rest from the second. To define f one needs the injections $\mathit{finl}_{n,m} : \mathit{Fin}\, n \to \mathit{Fin}(n + m)$ and $\mathit{finr}_{n,m} : \mathit{Fin}\, m \to \mathit{Fin}(n + m)$ and a non-standard eliminator for $\mathit{Fin}(n + m)$ that splits an element into its *finl* or *finr* component [39]. Proofs about $+\!\!+$ then proceed by that eliminator.

This approach captures the arithmetic of list functions elegantly, but the definitions and eliminators are non-trivial [39], each new function may need new ones, and — the decisive point — the obligations are statements about the families $\mathit{Fin}\, n$, to which arithmetic decision procedures do not apply. If instead we could capture the *behaviour* of the dependently typed reindexing maps by functions on $\mathbb{N}$, decidable arithmetic reasoning would become possible.

4.2 Towards non-dependent representations

The type $\mathit{Fin}\, n$ represents the set $\{m : \mathbb{N} \mid m < n\}$, and it can be translated to and from that set:

1. $\mathit{fnat} : \forall n.\, \mathit{Fin}\, n \to \mathbb{N}$ computes the value of an element: $\mathit{fnat}\, \mathit{fz} = 0$ and $\mathit{fnat}\,(\mathit{fs}\, i) = S\,(\mathit{fnat}\, i)$;
2. $\mathit{nfin} : \forall n\, m.\, m < n \to \mathit{Fin}\, n$ builds the element of value m (the case $n = 0$ is absurd, $m = 0$ gives fz, and $S\, m < S\, n$ gives $\mathit{fs}\,(\mathit{nfin}\, n\, m\ _)$ after peeling one successor off the proof);
3. $\mathit{finnat} : \forall n.\, \forall i : \mathit{Fin}\, n.\ \mathit{fnat}\, i < n$ proves that values are in range.

Observation 4.1. Writing $\mathit{Fin}\, n$ as $\{i \mid i < n\}$, a container morphism $(u, f) : (n : \mathbb{N} \triangleleft \{i \mid i < n\}) \Rightarrow (m : \mathbb{N} \triangleleft \{j \mid j < m\})$ is a pair

$$u : \mathbb{N} \to \mathbb{N}, \qquad f : \textstyle\prod_n \{i \mid i < u\, n\} \to \{j \mid j < n\},$$

and an element of $\{i \mid i < n\}$ is a pair (i, P) of a number i and a *proof* P that $i < n$. Equality (2) of two such morphisms (u, f) and (v, g) becomes

$$\frac{\forall n.\ u\, n = v\, n \qquad \forall n\, i\, j\, P\, Q.\ (i, P) \overset{jm}{=} (j, Q) \to f\, n\,(i, P) = g\, n\,(j, Q)}{(u, f) = (v, g)} \tag{3}$$

where $P : i < u\, n$ and $Q : j < v\, n$. Note that the hypothesis $(i, P) \overset{jm}{=} (j, Q)$ entails $i = j$; the proofs P and Q differ in type (one mentions u, the other v), which is why heterogeneous equality is still needed at this point.

Observation 4.2. Every reindexing map $f : \prod_n \mathit{Fin}(u\, n) \to \mathit{Fin}\, n$ has an *arithmetic shadow*: a partial function on numbers, defined on $i < u\, n$, that returns the value of f_n at the element of value i. We call it *reduce* f:

$$\mathit{reduce}\, f\, n\, i\, h \;=\; \mathit{fnat}\,(f_n\,(\mathit{nfin}\,(u\, n)\, i\, h)) \qquad (h : i < u\, n). \tag{4}$$

Given the proof h, *reduce* $f\, n\, i\, h$ is just a natural number, and *reduce* f corresponds exactly to f while being stated without finite types. Consequently, for every (u, f) there exists a total function $g : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ that agrees with the shadow of f wherever the latter is defined:

$$\forall n\, i\,(h : i < u\, n).\ g\, n\, i = \mathit{reduce}\, f\, n\, i\, h. \tag{5}$$

For reversal, $g\, n\, i = n{-}i{-}1$, and one proves once and for all that $\forall n\, i\, h.\ \mathit{reduce}\, \mathit{rv}\, n\, i\, h = n{-}i{-}1$.

	Goal	Step
(a)	$\forall n\, i\, j\, P\, Q.\ (i,P) \overset{jm}{=} (j,Q) \to f\, n\, (i,P) = g\, n\, (j,Q)$	second premise of (3), with $P : i < u\, n$, $Q : j < v\, n$
(b)	$\forall n\, i\, j\, P\, Q.\ (i,P) \overset{jm}{=} (j,Q) \to \mathit{reduce}\, f\, n\, i\, P = \mathit{reduce}\, g\, n\, j\, Q$	apply *fnat* to both sides (it is injective) and fold (4); the elements (i,P) are now written $\mathit{nfin}\, (u\, n)\, i\, P$
(c)	$\forall n\, i\, j\, P\, Q.\ (i,P) \overset{jm}{=} (j,Q) \to g_1\, n\, i = g_2\, n\, j$	rewrite with the shadow equations (5) for f and g
(d)	$\forall n\, i\, j\, P\, Q.\ i = j \to P \overset{jm}{=} Q \to g_1\, n\, i = g_2\, n\, j$	decompose the pair equation, using $u\, n = v\, n$: $(i,P) \overset{jm}{=} (j,Q) \leftrightarrow i = j \wedge P \overset{jm}{=} Q$
(e)	$\forall n\, i.\ i < u\, n \to i < v\, n \to g_1\, n\, i = g_2\, n\, i$	substitute $j := i$; the hypothesis $P \overset{jm}{=} Q$ is now vacuous (both are proofs of $i < u\, n$) and is discarded, leaving only the range conditions

Table 1: Evolution of the position obligation. Goal (e) mentions no finite types and no proofs.

How the goal evolves. Observation 4.2 lets us replace the second premise of (3) by arithmetic. Suppose we have already proved $u = v$ and are left with the position obligation. Table 1 traces the transformation of that obligation.

The upshot is that, provided reindexing maps are given by their shadows, equality of container morphisms between lists can be stated as

$$\frac{\forall n.\ u\, n = v\, n \qquad \forall n\, i.\ i < u\, n \to i < v\, n \to g_1\, n\, i = g_2\, n\, i}{(u, g_1) = (v, g_2)} \tag{6}$$

in place of (2). We call (6) *quasi-container* equality: it is sound with respect to Definition 2.10 (Theorem 4.9 below makes this precise), but it is stated purely in arithmetic. If u, v, g_1 and g_2 lie in a decidable fragment of arithmetic, (6) can be decided. Which fragment? The shape maps of *crev* (identity) and *cappn* (addition) suggest linear arithmetic, and we now show that this suggestion is right.

4.3 Presburger-definable morphisms

Presburger integer arithmetic (PIA) is the first-order theory of the structure $\langle \mathbb{Z}, 0, 1, +, \leq \rangle$; its restriction to the natural numbers is *Presburger natural arithmetic* (PNA). Presburger showed that PIA is decidable [36], and the decidability of PNA follows (interpret $\mathbb{N}$ as the non-negative integers). Cut-off subtraction and conditionals on linear inequalities are definable in PNA: the graph of $r = a \mathbin{\dot{-}} b$ is $(b \leq a \wedge r + b = a) \vee (a < b \wedge r = 0)$, and the graph of $r = (\text{if } a < b \text{ then } x \text{ else } y)$ is $(a < b \wedge r = x) \vee (b \leq a \wedge r = y)$. We shall use this repeatedly.

Definition 4.3 (Linear functions)**.** A function $F : \mathbb{N}^k \to \mathbb{N}$ is *linear definable* if it is denoted by a term of the grammar

$$\mathit{lfun} \ ::= \ \mathit{const}\, c \ \mid \ \mathit{var}\, j \ \mid \ \mathit{add}\, \mathit{lfun}\, \mathit{lfun} \qquad (c \in \mathbb{N},\ j < k)$$

interpreted in the evident way: $\mathit{const}\, c$ is the constant function, $\mathit{var}\, j$ is the j-th projection and *add* is pointwise addition. It is *piecewise-linear definable* if it is denoted by a term of the extended grammar

$$\mathit{pl} \ ::= \ \mathit{lp}\, \mathit{lfun}' \ \mid \ \mathit{pf}\, \mathit{pl}\, \mathit{pl}\, \mathit{pl}, \qquad \mathit{lfun}' \ ::= \ \mathit{lfun} \ \mid \ \mathit{sub}\, \mathit{lfun}'\, \mathit{lfun}'$$

where *sub* is cut-off subtraction and $\mathit{pf}\, c\, l\, r$ denotes "if $0 < c$ then l else r".

Since a condition $a < b$ is the same as $0 < b \dot{-} a$, nested *pf* expresses any finite case analysis on linear inequalities; the grammar is that of the Coq types `lfun` and `Plfun` of §4.6. By the remarks preceding the definition, the graph of every piecewise-linear definable function is given by a quantifier-free formula of PNA ([Coq] `suB_graph`, `pf_graph`).

Definition 4.4 (Linear and Presburger morphisms)**.** Let $(u, f) : (\mathbf{n} : \mathbb{N}^k \triangleleft \mathit{Fin}\, n_1 + \cdots + \mathit{Fin}\, n_k) \Rightarrow (\mathbb{N} \triangleleft \mathit{Fin})$ be a container morphism from k lists to a list.

(i) (u, f) is *linear* if u is linear definable.

(ii) (u, f) is *Presburger* if u and the shadow of f are piecewise-linear definable, where positions of the k inputs are numbered consecutively, so that the shadow is a function $\mathbb{N}^k \times \mathbb{N} \to \mathbb{N}$ (output position i to a position in $\mathit{Fin}(n_1 + \cdots + n_k)$).

Cartesian morphisms are sometimes called linear in the literature [3]; our use of "linear" is different (it refers to the shape map), and we reserve "cartesian" for Definition 2.7. *crev* and *cappn* are linear, with shape maps $\mathit{var}\, 0$ and $\mathit{add}\, (\mathit{var}\, 0)\, (\mathit{var}\, 1)$ respectively.

Proposition 4.5. *The shadow of the reindexing map of a linear morphism is piecewise-linear definable.*

Proof sketch. By induction on the term denoting u. For *const* and *var* the shadow is obtained directly by *reduce*. For $\mathit{add}\, l\, r$, an output position $i < u\, \mathbf{n}$ satisfies either $i < l\, \mathbf{n}$ or $i - l\, \mathbf{n} < r\, \mathbf{n}$, by the lemma $\mathit{leplus_or} : \forall i\, n\, m.\ i < n + m \to \{i < n\} + \{i - n < m\}$; in the first case the position is routed through the *finl* component and in the second through the *finr* component, recursively. Each split introduces one breakpoint, linear in the n_j, so the result is piecewise linear. Full details are in [38, Prop. 6.2]. □

Linear morphisms do not cover *tail*, *head*, *last* or *but_last*, whose shape maps involve subtraction, nor *take*, whose shape map is a minimum. All of these are Presburger morphisms, however, and in fact the Presburger morphisms are closed under the operation that matters.

Proposition 4.6 (Closure under composition)**.** *If (u_1, g_1) and (u_2, g_2) are Presburger morphisms, so is their composite.*

Proof. By Definition 2.8 the composite has shape map $u_2 \circ u_1$ and shadow $\lambda \mathbf{n}\, i.\ g_1\, \mathbf{n}\, (g_2\, (u_1\, \mathbf{n})\, i)$; piecewise-linear terms are closed under substitution of terms for variables, conditionals being propagated outwards ([Coq] `psubst`, `psubst_ok`). □

Theorem 4.7 (Decidability)**.** *Let (u, g_1) and (v, g_2) be Presburger morphisms. The equality* (6) *between them is decidable. So are the* range condition $\forall \mathbf{n}\, i.\ i < u\, \mathbf{n} \to g_1\, \mathbf{n}\, i < n_1 + \cdots + n_k$*, and, for $k = 1$, the* losslessness condition

$$\forall n.\ u\, n = n \ \wedge\ \forall i\, j.\ i < n \to j < n \to g_1\, n\, i = g_1\, n\, j \to i = j.$$

Proof. Let $\gamma_u(\mathbf{n}, r)$, $\gamma_v(\mathbf{n}, r)$, $\gamma_1(\mathbf{n}, i, r)$ and $\gamma_2(\mathbf{n}, i, r)$ be the quantifier-free PNA formulae defining the graphs of u, v, g_1 and g_2. Then (6) is equivalent to the PNA sentence

$$\begin{aligned} &\forall \mathbf{n}\, r\, r'.\ \gamma_u(\mathbf{n}, r) \to \gamma_v(\mathbf{n}, r') \to r = r' \\ \wedge\ &\forall \mathbf{n}\, i\, r\, r_1\, r_2.\ \gamma_u(\mathbf{n}, r) \to i < r \to \gamma_1(\mathbf{n}, i, r_1) \to \gamma_2(\mathbf{n}, i, r_2) \to r_1 = r_2, \end{aligned}$$

which is decidable by Presburger's theorem. The other two conditions are translated in the same way. □

The proof shows more than decidability. All three sentences are *universal*: a block of $\forall$s in front of a quantifier-free matrix. Deciding such a sentence amounts to deciding the unsatisfiability of a quantifier-free formula, a problem that is NP-complete rather than doubly exponential, and that is exactly what the Omega test [40], Coq's `lia` and SMT solvers are built for. This matters for the practicality discussed in §4.6. By contrast the dual of losslessness, surjectivity of the position maps ($\forall n\, j.\, j < n \to \exists i < u\, n.\, g_1\, n\, i = j$), has an existential quantifier and lies outside the fragment; happily, for finite sets injectivity together with $u\, n = n$ already implies bijectivity, which is why the losslessness condition is stated as it is.

4.4 Representing Presburger morphisms

We now fix the concrete representation used in the development. A Presburger morphism of *arity* k (from k lists to a list) is a pair of piecewise-linear terms

$$\mathsf{PPCm} \;=\; pl \times pl,$$

the first denoting the shape map over the variables $0, \ldots, k-1$ (the input lengths), the second denoting the shadow over the variables $0, \ldots, k$, where variable k is the output position i. Positions of the k inputs are numbered consecutively ("flat" encoding), via the isomorphism $\mathit{Fin}\, n_1 + \cdots + \mathit{Fin}\, n_k \cong \mathit{Fin}(n_1 + \cdots + n_k)$.

Example 4.8. Writing x_0, x_1, x_2 for the variables,

$$\begin{array}{ll} \mathit{Rev} = (x_0,\ (x_0 \mathbin{\dot-} x_1) \mathbin{\dot-} 1) & \mathit{Tail} = (x_0 \mathbin{\dot-} 1,\ x_1 + 1) \\ \mathit{Take}_m = (\text{if } x_0 < m \text{ then } x_0 \text{ else } m,\ x_1) & \mathit{Drop}_m = (x_0 \mathbin{\dot-} m,\ x_1 + m) \\ \mathit{App} = (x_0 + x_1,\ x_2) & \mathit{last}_k = (\text{if } x_0 < k \text{ then } x_0 \text{ else } k,\ x_1 + (x_0 \mathbin{\dot-} k)). \end{array}$$

Observe that with the flat encoding the position map of *App* is the identity: output position i is input position i of the concatenation.[2]

The *interpretation* of $(u, g) : \mathsf{PPCm}$ of arity k on inputs $(n_1, h_1), \ldots, (n_k, h_k)$, where $h_j : \mathbb{N} \to X$ is the lookup function of the j-th list, is

$$[\![(u, g)]\!]\ \big((n_1, h_1), \ldots, (n_k, h_k)\big) \;=\; \big(u\, \mathbf{n},\ \lambda i.\ \mathit{flat}\,(g\, \mathbf{n}\, i)\big), \tag{7}$$

where $\mathit{flat}\, p = h_1\, p$ if $p < n_1$, $\mathit{flat}\, p = h_2\, (p - n_1)$ if $n_1 \le p < n_1 + n_2$, and so on ([Coq] `interp`, `flat`). This is *mmap* of (1) for the flat encoding; a default value is returned for an out-of-range position, which a morphism satisfying the range condition of Theorem 4.7 never produces.

Theorem 4.9 (Soundness of quasi-container equality)**.** *[Coq: `pcmEQ_ok`] If (u, g_1) and (v, g_2) satisfy* (6)*, then for all inputs their interpretations have the same length and the same element at every position.*

Definition 4.10 (Composition of representations)**.** Let $l = (u_l, g_l)$ have arity 1 and $r = (u_r, g_r)$ have arity k. Their composite $\mathit{comp}_1\, l\, r$, of arity k, is

$$\big(\ u_l[x_0 := u_r],\ \ g_r[x_k := g_l[x_0 := u_r,\ x_1 := x_k]]\ \big),$$

where $t[x := s]$ is substitution of piecewise-linear terms. Let l have arity 2 and $r_1 = (u_1, g_1)$, $r_2 = (u_2, g_2)$ have arity k *over the same k inputs*. Their composite $\mathit{comp}_2\, l\, r_1\, r_2$, of arity k, has shape map $u_l[x_0 := u_1, x_1 := u_2]$ and, with $q = g_l[x_0 := u_1, x_1 := u_2, x_2 := x_k]$ the position in the intermediate concatenation, position map

$$\text{if } q < u_1 \text{ then } g_1[x_k := q] \text{ else } g_2[x_k := q \mathbin{\dot-} u_1].$$

[2] A position map that subtracts n in the second case, as one might first write, double counts the offset that the interpretation already applies; see §4.6.

The first clause is Definition 2.8 written out for terms. The second is the composition needed for a binary function applied to two functions of the same argument(s) — for instance $\mathit{take}_m\, b +\!\!+ \mathit{drop}_m\, b$, or $\mathit{rev}\, ys +\!\!+ \mathit{rev}\, xs$ as a function of (xs, ys) — and it is where the flat encoding earns its keep: the position map is a case split on which intermediate list the position falls in. (In the development the terms substituted for u_r, u_1, u_2 are first *shielded*, that is, their variables $\geq k$ are set to 0, so that a term of arity k cannot observe the output-position variable of the composite; this makes the following theorem exact rather than subject to a free-variable side condition.)

Theorem 4.11 (Correctness of composition)**.** *[Coq: `comp1_ok`, `comp2_ok`] If l satisfies the range condition, then for all inputs $\mathbf{x}$,*

$$[\![\mathit{comp}_1\, l\, r]\!]\, \mathbf{x} = [\![l]\!]\, ([\![r]\!]\, \mathbf{x})$$
$$[\![\mathit{comp}_2\, l\, r_1\, r_2]\!]\, \mathbf{x} = [\![l]\!]\, ([\![r_1]\!]\, \mathbf{x}, [\![r_2]\!]\, \mathbf{x}),$$

up to equality of lengths and of elements at every position.

Equality (6) is implemented as a proposition `pcmEQ` and decided by the tactic `containers` described in §4.6.

4.5 A closer look at some functions

The shape map of a function in this setting is exactly the shape map of its container morphism, while its position map is the shadow of the container morphism's reindexing map. To represent a function one therefore states what its container morphism does to lengths and indices, and the representation can usually be read off a whiteboard.

***take* and *drop*.** take_m retains the first m elements: if $m < n$ the output has length m, otherwise n, and output position i is always input position i. drop_m removes the first m elements: the output has length $n \mathbin{\dot{-}} m$ and output position i is input position $i + m$. Hence the representations in Example 4.8. Note that take_m is *not* cartesian: when $m < n$ positions $m, \ldots, n-1$ of the input are dropped.

***head*, *last*, *but_last* and *tail*.** As functions $\mathit{List}\, X \to \mathit{List}\, X$ (returning a list of length at most one, to stay polymorphic), *head* and *last* have shape map $n \mathbin{\dot{-}} (n \mathbin{\dot{-}} 1)$, which is 1 for $n > 0$ and 0 for $n = 0$; the single output position is input position 0 for *head* and $n - 1$ for *last*. *but_last* has shape map $n \mathbin{\dot{-}} 1$ and identity position map; *tail* has the same shape map and position map $i + 1$.

Rotation. *rot* moves the first element to the back and *unrot* moves the last element to the front. Both keep the length. Output position i of *rot* is input position $i + 1$ when $i < n - 1$ and 0 otherwise; output position i of *unrot* is input position $i - 1$ when $i > 0$ and $n - 1$ otherwise. In the dependently typed representation these require the view of $\mathit{Fin}(S\, n)$ as *top-or-emb* (Example 2.5); here they are two conditionals.

Figure 4 lists properties that are proved automatically. Several of them were beyond the scope of the ellipsis technique [14, 37], and all of them are proved without the subsidiary lemmas and eliminators that the dependently typed system of [39] required.

4.6 The Coq development

The updated development consists of four files, compiles with Coq 8.18 and the Rocq Prover, uses only the standard library, and contains no axioms or admitted lemmas.

$$rev\,(rev\,xs) = xs \qquad drop_1\,(unrot\,xs) = but_last\,xs$$
$$rev\,(xs +\!\!+ ys) = rev\,ys +\!\!+ rev\,xs \qquad rot\,(unrot\,xs) = xs$$
$$head\,(rev\,xs) = last\,xs \qquad unrot\,(rot\,xs) = xs$$
$$tail\,(rev\,xs) = rev\,(but_last\,xs) \qquad head\,(rev\,(rot\,xs)) = head\,xs$$
$$drop_n\,(drop_m\,xs) = drop_{n+m}\,xs \qquad last\,(rot\,xs) = head\,xs$$
$$drop_n\,(take_m\,xs) = take_{m-n}\,(drop_n\,xs) \qquad head\,(unrot\,xs) = last\,xs$$
$$take_n\,(drop_m\,xs) = drop_m\,(take_{n+m}\,xs) \qquad but_last\,(rot\,xs) = tail\,xs$$

Figure 4: Properties of lists proved automatically by the tactic `containers` ([Coq] file `Examples.v`). Each is stated as a `pcmEQ` between composites of the representations of Example 4.8.

`Piecewise.v`. The grammar of Definition 4.3 is the pair of inductive types

```
Inductive lfun : Set :=
  | cnst : nat -> lfun
  | vr   : nat -> lfun
  | adD  : lfun -> lfun -> lfun
  | suB  : lfun -> lfun -> lfun.
```

```
Inductive Plfun : Set :=
  | lp : lfun -> Plfun
  | pf : Plfun -> Plfun -> Plfun -> Plfun.
```

with evaluation functions `evl` and `pevl` against an environment $\rho : \mathbb{N} \to \mathbb{N}$ (variable j reads $\rho\, j$). Variables are plain numbers rather than elements of `Fin n`, which removes every dependent pattern match from the arithmetic core. Substitution `psubst` is defined with conditionals propagated outwards, and proved correct.

`ListMorphisms.v`. A morphism is a pair `PPCm := Plfun * Plfun`; its arity is supplied to the equality and the interpretation rather than carried in the type. The file defines the interpretation (7), the equality

```
Definition pcmEQ (k : nat) (l r : PPCm) : Prop :=
  forall rho, pevl (shape l) rho = pevl (shape r) rho /\
    forall i, i < pevl (shape l) rho ->
      pevl (pos l) (ext rho k i) = pevl (pos r) (ext rho k i).
```

(where `ext rho k i` binds variable k to i), its soundness (Theorem 4.9), the range and losslessness conditions, composition (Definition 4.10) with its correctness (Theorem 4.11), and the tactic

```
Ltac containers := unfold_morphism; arith.
```

Here `unfold_morphism` unfolds the definitions, splits the conjunction, evaluates the terms symbolically (rewriting with the correctness lemmas for substitution) and `arith` eliminates every remaining conditional by case analysis and calls `lia`. No pre-processing of the kind the earlier `omega`-based tactic needed (discarding redundant hypotheses, ad hoc inequality reasoning) is required.

`Examples.v`. The representations of Example 4.8 and the properties of Figure 4, each proved by `containers`. A representation is only as good as its agreement with the function it is meant to represent — the earlier development contained a *take* whose shape map was $\max(n, m)$ rather than $\min(n, m)$, a swapped conditional that no property in Figure 4 detects — so each basic morphism is validated against the standard library by a *transfer lemma*, e.g.

```
Lemma Take_spec : forall m (l : list X),
    RepEq (interp (Take m) [of_list d l] d) (of_list d (firstn m l)).
```

Statement	[38]	this version
$rev\,(rev\,xs) = xs$	6 s	0.02 s
$take_n\,(drop_m\,xs) = drop_m\,(take_{n+m}\,xs)$	—	0.02 s
$head\,(rev\,(rot\,xs)) = head\,xs$	—	0.04 s
$rev\,(xs +\!\!+ ys) = rev\,ys +\!\!+ rev\,xs$	not provable†	0.18 s
$take_m\,b +\!\!+ drop_m\,b = b$	—	0.06 s
$last_k\,(h +\!\!+ t) = last_k\,(last_k\,h +\!\!+ t)$	—	0.43 s

Table 2: Time taken by the tactic (Coq 8.18, Intel Xeon 2.1 GHz). The figure for [38] is from its §6.3.2 (Intel E5200). †Not expressible in the development of [38], which lacked the composition $comp_2$.

where `of_list` turns a list into a length-and-lookup pair and `RepEq` is equality of length and of elements at every position. How hard is it to relate the container-style definitions to the usual recursive ones? Each transfer lemma is a short structural induction (between three and ten lines; `Rev_spec` uses the library lemma `rev_nth`), and it is done once per function, not once per property. With the transfer lemmas in hand, theorems about ordinary lists follow from the decision procedure *with no induction at all*: the file derives, among others,

```
Theorem rev_involutive_via_containers : forall l : list X, rev (rev l) = l.
Theorem firstn_skipn_via_containers : forall m (l : list X), firstn m l ++ skipn m
    l = l.
```

by chaining the transfer lemmas, Theorem 4.11, Theorem 4.9 and the relevant `pcmEQ`. This is a "two-tier" methodology: one recursive definition per function for programming, one container representation per function for proving, connected once.

`Containers.v`. Containers, extensions, morphisms, *mmap* with its functoriality, cartesian morphisms, agents, wrapping and runs, exactly as in §2–3; the tree container and the proof that flattening is cartesian (§3.4); and the bridge to the arithmetic representation described in §5.

Performance. Table 2 gives proof times. For comparison, the `omega`-based development of [38] took 6 s for $rev \circ rev = id$. The present development proves every property of Figure 4 in at most 0.2 s (all but one in under 40 ms), and the most expensive statement in the development (the sliding-window invariant of §5, which composes an arity-2 morphism) in 0.43 s. Three things account for the speed: `lia` (Micromega) replaces `omega`, which was deprecated in Coq 8.12 and removed in 8.14; the non-dependent, nat-indexed terms give `lia` small goals with few atoms; and the universal form of the statements (Theorem 4.7) puts them in the quantifier-free fragment for which these procedures are designed.

Limitations. The procedure handles polymorphic functions $List(X)^k \to List(X)$ whose container morphisms are Presburger. Functions on nested lists, and functions into other data types, are outside its scope; §7 discusses extensions.

5 Verifying List-Shaped Agent Adapters

We now combine §3 and §4. Throughout, "adapter" means an adapter between list-shaped interfaces, represented as in §4.4.

5.1 From arithmetic equality to agent behaviour

The arithmetic representation lives on natural numbers; the agents of Definition 3.4 live on the list container with its dependently typed positions. The two are connected by the range condition.

Theorem 5.1. *[Coq: `to_cmor`, `to_cmor_interp`, `pcmEQ_wrap`]*

(i) Every representation $m = (u, g)$ of arity 1 satisfying the range condition determines a container morphism $\widehat{m} : (\mathbb{N} \triangleleft Fin) \Rightarrow (\mathbb{N} \triangleleft Fin)$, with shape map u and position map $f_n\, i = g\, n\, i$ (the range condition supplies the proof that $g\, n\, i < n$).

(ii) The extension of $\widehat{m}$ agrees with the interpretation (7) of m: reading a list off $mmap\, \widehat{m}\, e$ gives the same length and elements as applying $[\![m]\!]$ to the list read off e.

(iii) Consequently, if l and r satisfy the range condition and $pcmEQ\, l\, r$ holds, then for every agent a on the list interface and every state x, the one-step behaviours $wrap\, \widehat{l}\, a\, x$ and $wrap\, \widehat{r}\, a\, x$ have the same move and the same continuation at every response; by Proposition 3.6(iii) the wrapped agents have the same runs.

Part (iii) is the statement that an equation decided by `containers` is a theorem about *behaviour*: whatever the agent — an LLM-backed planner, a retriever, a hand-written state machine — the two adapters make it indistinguishable to its environment.

5.2 Losslessness

An adapter that drops or duplicates responses is sometimes intended ($last_k$ must drop old turns) and sometimes a bug (a fan-in that loses a tool result). Either way it is a fact worth knowing, and for governance and auditability it is a fact worth *checking*.

Proposition 5.2. *[Coq: `lossless` and the lemmas `*_lossless`, `*_lossy`] Losslessness of an adapter — cartesianness of the corresponding container morphism — is decidable (Theorem 4.7). Reversal and the two rotations are lossless; tail, $take_m$, $drop_{m+1}$ and $last_k$ are not, and the procedure exhibits a witness in each case (a history of length $k+1$ for $last_k$).*

5.3 Worked examples

All statements in this section are from the file `Examples.v`, where each is proved or refuted by the tactic `containers` in well under a second.

Context windows. Let h be a history and t the new turns to be appended. A sliding-window implementation must decide whether to truncate before or after appending. The procedure proves that it does not matter:

$$last_k\,(h +\!\!+ t) \;=\; last_k\,(last_k\, h +\!\!+ t), \tag{8}$$

so a system may keep only the last k turns between calls without ever diverging from a system that keeps everything and truncates on demand. In the representation, (8) is the equality of arity-2 morphisms

$$comp_1\, last_k\, App \;=\; comp_1\, last_k\, \big(comp_2\, App\, (last_k \text{ on } x_0)\, (id \text{ on } x_1)\big)$$

([Coq] `lastk_app`). Two further invariants are proved in the same way: truncation is idempotent, $last_k\,(last_k\, h) = last_k\, h$, and nested windows collapse to the tighter one, $last_k\,(last_{k+1}\, h) = last_k\, h$.

A refuted law. It is tempting to "optimise" the window by dropping the oldest turn of a $(k+1)$-window, on the grounds that $drop_1\,(last_{k+1}\,h) = last_k\,h$. This is false when the history is shorter than the window: for a history of length 1 and $k = 1$ the left side is empty and the right side is not. The procedure refutes it with exactly that witness ([Coq] `drop_lastk_refuted`). Likewise an off-by-one position map, $f_n\,i = i + (n \dot{-} k) + 1$, is rejected with the witness $n = k+1$, $i = 0$ (`badlastk_detected`). A false adapter law is a routing bug: the agent would be shown the wrong turn. That the same procedure both proves the true laws and refutes the false ones with counterexamples is, for this application, the essential property.

Fan-out and fan-in. An orchestrator that splits a batch of n requests at index m and reassembles the n results must restore the original correspondence between requests and results. The procedure proves $take_m\,b +\!\!+ drop_m\,b = b$ as an equation between the arity-1 morphisms $comp_2\;App\;Take_m\;Drop_m$ and id ([Coq] `split_merge`): result i is routed back to request i. Together with Proposition 3.6(ii) this scales: a tree of splits and merges is a composite of adapters, and its equality to the identity is one `pcmEQ`.

Hierarchical planners. A planner on the tree interface of §3.4, wrapped by *flatten*, is an agent on the batch interface, and the results above apply to it unchanged. In particular, a plan with n leaves dispatched to two worker pools by splitting at m and reassembled has, by `split_merge` and Proposition 3.6(ii), the adapter *flatten* itself, so each leaf receives exactly its own result; and because *flatten* is cartesian (Proposition 3.8) the whole pipeline from plan to pooled results is lossless. The development packages the first step as `flat_planner`.

Reorderings. The identities of Figure 4 read, in the agent setting, as facts about rankings and histories: "the top candidate of the reversed ranking is the bottom candidate of the original" ($head \circ rev = last$), "reversing twice is the identity", "rotating a round-robin schedule forward and then back changes nothing" ($unrot \circ rot = id$).

5.4 Generate and check

It would be interesting to consider whether the translation from a recursive definition to its container representation could be automated. Two observations bear on this. First, the thesis [38, §7.2.3] already gives a *semantic* translation: for any polymorphic $F : \forall X.\, List\,X \to List\,X$, Theorem 2.9 yields its container morphism, and *reduce* (4) yields the shadow; what is not automatic is recognising the shadow as a piecewise-linear *term*, which is a normalisation problem rather than a proof problem. Second, the division of labour between an untrusted *proposer* and a trusted *checker* has become the standard architecture for combining language models with proof assistants [23, 5], and it fits here exactly: a model may propose the representation (u, g) of a function together with its transfer lemma, and the transfer lemma's proof (a short induction) and all subsequent properties (decided by `containers`) are checked by Coq. Ghani's remark that a language model is "like a precocious PhD student" who "talks nonsense quite often" [20] is precisely the reason the checker must be a decision procedure and not another model. A natural objection to the container style — that index arithmetic is an unnatural way to program lists, closer to arrays in C than to lists in a functional language — is in this domain inverted: index arithmetic on histories and batches (`history[-k:]`, `results[i]`) is how orchestration code is written today. What is missing is the proof that the arithmetic is right, which is what the procedure supplies.

5.5 What this does and does not establish

The results above are about the composition of components along typed, list-shaped interfaces. They say nothing about whether a language model will produce a parseable output (that is the

parser's job), nothing about the model's internals, and nothing about interfaces whose shape is not a count. Within their scope they deliver, for the list fragment, the three properties that Kodamai's "math-first" programme asks of enterprise agents [25]: adapters are verified rather than tested ("error-proof"), interfaces are typed contracts specific to the data flowing through them ("specialised"), and a component can be swapped for another with the same interface, with the equations between adapters guaranteeing that the swap is behaviour-preserving ("adaptive"). Extending the decidable fragment beyond lists — to trees, to interfaces indexed by several counts, to the mode-dependent interfaces of [42] — is the natural next step, and §7 outlines it.

6 Related Work

Containers and polynomial functors. Containers were introduced by Abbott, Altenkirch and Ghani [1, 2, 3] as a semantic account of strictly positive data types, and extended to indexed containers in [6]. Polynomial functors, the same objects in a categorical dress, were studied by Gambino and Kock [18] and taken up by Spivak as a setting for "mode-dependent dynamics" [41]; the monograph of Niu and Spivak [34] develops the category **Poly** systematically, including the reading of lenses as interfaces of interacting systems that §3 adopts. Libkind and Spivak [28] use polynomial functors to model hierarchical task delegation between agents, with morphisms as delegations; our adapters are the special case of list-shaped interfaces, and our contribution is the decision procedure for that case. Spivak's recent work on interfaces that change during interaction [42] suggests where a richer decidable fragment might be sought. Lenses as bidirectional transformations go back to the view-update problem; their use in compositional game theory [19] and categorical cybernetics [15] is the nearest precedent for "adapters that route information backwards".

Containers in reasoning. Containers were used to construct polymorphic programs with quotient types in [4], and as the underpinning of the ellipsis technique in [39], of which the present paper is a continuation. Generic programming with containers and universes is treated in [7, 33].

Typed agentic AI. The gap between untyped LLM output and typed components is addressed in practice by structured-output APIs that constrain generation to a JSON schema [35] and by tool protocols such as the Model Context Protocol [29], which type tool invocations and results. Kodamai's Calvin Grove platform [25] composes agents as typed microservices whose interfaces are "contracts", with the stated aim of correctness by construction; Ghani describes its type system as built on containers, with agents as first-class values that can be passed to and returned by other agents [20]. The present paper supplies a decision procedure for one fragment of such a type system. On the proof side, Harmonic's Aristotle [23] and DeepMind's AlphaProof [5] produce Lean proofs checked by the kernel; §5.4 places our procedure in the same proposer/checker architecture.

Decision procedures for data structures. Suter, Dotta and Kuncak [43] extend quantifier-free constraints on algebraic data types with catamorphisms, allowing reasoning about the relationship between a data structure and the value of a recursive function applied to it. Zhang, Sipma and Manna [44] combine recursive data structures with Presburger arithmetic through size functions. Bouajjani et al. [11] define a logic for composite data structures with a decidable satisfiability problem. Ghilardi et al. [21] extend the theory of arrays with Presburger-definable indices; Bradley, Manna and Sipma [12] identify a decidable fragment of the theory of arrays, extended to infinite arrays by Habermehl, Iosif and Vojnar [22] using automata [10]. Arons et al. [9] verify parameterized systems by automatically computed inductive assertions. Our

procedure differs from all of these in its object: not the values stored in a structure but the *polymorphic* functions between structures, obtained through the representation theorem; the arithmetic is on lengths and indices alone.

Presburger arithmetic in verification. Presburger's theorem [36] and its rational analogue [26] underlie a large number of tools. The Omega test [40] has been used for timing verification [8] and datapath verification [13, 24]. In proof assistants, Coq's `omega` has been superseded by the certificate-checking `lia` of the Micromega plugin, and Lean 4 acquired an `omega` decision procedure in 2024 [27]; the procedure of this paper is independent of the choice and would port to Lean directly. The canonical form for piecewise-linear functions of Chua and Kang [17], used in [38, Ch. 5] to decide equality of piecewise-linear functions over ordered fields, is not needed here because the statements of Theorem 4.7 are handled directly by the arithmetic procedure; Carette [16] gives canonical forms for more general piecewise functions.

7 Conclusion and Further Work

We have shown how the container representation of polymorphic functions between lists can be reduced to arithmetic on natural numbers, how equality of such functions becomes a universal sentence of Presburger arithmetic when the shape and position maps are piecewise linear, and how the resulting decision procedure proves the usual identities between list functions without induction. We have then read the same mathematics as a theory of typed agent interfaces: container morphisms are adapters, wrapping is functorial, and for list-shaped interfaces the equality and losslessness of adapters are decidable, with behavioural consequences for every agent so wrapped. The accompanying development is new, axiom-free, and fast.

Several directions remain.

Nested lists and other data types. The procedure handles $\mathit{List}(X)^k \to \mathit{List}(X)$ only. For nested lists the shape of the inner lists is a vector of lengths; extending the term language so that *var* ranges over a vector of vectors is straightforward syntactically, but the resulting position maps need quantification over the inner index and may leave the universal fragment. For other data types a promising starting point is the data types with a *cartesian* morphism into lists, of which flattening (§3.4) is the paradigm. Through such a morphism, reindexing maps between trees may be representable as reindexing maps between lists, and hence decided: an equation $m = m'$ between tree adapters whose position maps are determined by their flattenings would reduce to an equation between list adapters, and the question is for which classes of tree adapters this reduction is complete.

Full Presburger arithmetic. Properties with existential content — surjectivity of a position map, or the existence of a split point with some property — leave the universal fragment. They remain decidable, by Cooper's algorithm or by automata-based procedures, at a higher cost. An implementation along the lines of [22] would extend the range of adapter properties that can be checked.

Higher-order adapters. **Poly** is monoidal closed [34], so the adapters between two interfaces form an interface $[I, J]$, and "meta-agents" that take agents as inputs and return agents [20] are agents on such interfaces. Our procedure decides equality in the list-shaped, Presburger fragment of $[I, J]$; a natural question is whether a decidable fragment of the full internal hom can be identified, which would make the typing of meta-agents checkable rather than merely expressible.

Mode-dependent interfaces. Interfaces whose positions depend on the history of the interaction [42] are the general case of a context window: the window's shape at step t depends on everything emitted so far. The window invariants of §5.3 are the first examples of equations between such interfaces that can be decided; characterising the decidable fragment is open.

Portability. Nothing in the development depends on Coq specifically. A Lean 4 port, using its `omega` tactic, would bring the procedure to the proof assistant in which most LLM-generated proofs are currently checked.

Acknowledgements

I am grateful to Neil Ghani for suggesting that this work be revisited in the light of agentic AI and for the conversations that shaped §3, and to Thorsten Altenkirch and Conor McBride for their part in the work on which §4 builds.

References

[1] M. Abbott. *Categories of Containers*. Ph.D. thesis, University of Leicester, 2003.

[2] M. Abbott, T. Altenkirch and N. Ghani. Categories of containers. In A. Gordon (ed.), *Proceedings of FOSSACS 2003*, LNCS 2620, pp. 23–38. Springer, 2003.

[3] M. Abbott, T. Altenkirch and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science* 342(1):3–27, 2005.

[4] M. Abbott, T. Altenkirch, N. Ghani and C. McBride. Constructing polymorphic programs with quotient types. In *Mathematics of Program Construction (MPC 2004)*, LNCS 3125. Springer, 2004.

[5] AlphaProof and AlphaGeometry teams. AI achieves silver-medal standard solving International Mathematical Olympiad problems. Google DeepMind, July 2024.

[6] T. Altenkirch, N. Ghani, P. Hancock, C. McBride and P. Morris. Indexed containers. *Journal of Functional Programming* 25, e5, 2015.

[7] T. Altenkirch, C. McBride and P. Morris. Generic programming with dependent types. In *Datatype-Generic Programming*, LNCS 4719, pp. 209–257. Springer, 2007.

[8] T. Amon, G. Borriello, T. Hu and J. Liu. Symbolic timing verification of timing diagrams using Presburger formulas. In *DAC '97*, pp. 226–231. ACM, 1997.

[9] T. Arons, A. Pnueli, S. Ruah, Y. Xu and L. Zuck. Parameterized verification with automatically computed inductive assertions. In *CAV 2001*, LNCS 2102, pp. 221–234. Springer, 2001.

[10] J. R. Büchi. Weak second-order arithmetic and finite automata. *Mathematical Logic Quarterly* 6:66–92, 1960.

[11] A. Bouajjani, C. Drăgoi, C. Enea and M. Sighireanu. A logic-based framework for reasoning about composite data structures. In *CONCUR 2009*, LNCS 5710, pp. 178–195. Springer, 2009.

[12] A. R. Bradley, Z. Manna and H. B. Sipma. What's decidable about arrays? In *VMCAI 2006*, LNCS 3855, pp. 427–442. Springer, 2006.

[13] R. Brinkmann and R. Drechsler. RTL-datapath verification using integer linear programming. In *ASP-DAC '02*, p. 741. IEEE, 2002.

[14] A. Bundy and J. Richardson. Proofs about lists using ellipsis. In *Logic Programming and Automated Reasoning (LPAR '99)*, LNCS 1705. Springer, 1999.

[15] M. Capucci, B. Gavranović, J. Hedges and E. F. Rischel. Towards foundations of categorical cybernetics. In *Applied Category Theory 2021*, EPTCS 372, pp. 235–248, 2022.

[16] J. Carette. A canonical form for piecewise defined functions. In *ISSAC '07*, pp. 77–84. ACM, 2007.

[17] L. O. Chua and S. M. Kang. Section-wise piecewise-linear functions: canonical representation, properties, and applications. *Proceedings of the IEEE* 65(6):915–929, 1977.

[18] N. Gambino and J. Kock. Polynomial functors and polynomial monads. *Mathematical Proceedings of the Cambridge Philosophical Society* 154(1):153–192, 2013.

[19] N. Ghani, J. Hedges, V. Winschel and P. Zahn. Compositional game theory. In *LICS 2018*, pp. 472–481. ACM, 2018.

[20] N. Ghani, interviewed by T. Altenkirch and D. Sarikaya. Category theory meets agentic AI. *About Logic* podcast, 23 September, `https://www.youtube.com/watch?v=FbtlLc352fs`, 2026.

[21] S. Ghilardi, E. Nicolini, S. Ranise and D. Zucchelli. Decision procedures for extensions of the theory of arrays. *Annals of Mathematics and Artificial Intelligence* 50:231–254, 2007.

[22] P. Habermehl, R. Iosif and T. Vojnar. What else is decidable about integer arrays? In *FOSSACS 2008*, LNCS 4962, pp. 474–489. Springer, 2008.

[23] Harmonic. Aristotle: gold-medal performance at the International Mathematical Olympiad with formally verified Lean proofs. Technical report, arXiv:2510.01346, October 2025.

[24] P. Johannsen and R. Drechsler. Formal verification on the RT level computing one-to-one design abstractions by signal width reduction. In *VLSI-SoC 2001*, pp. 127–132, 2001.

[25] M. Achour, interviewed by A. Dalton. Math-first AI: how Kodamai embeds verification and trust. *EE Journal*, Amelia's Weekly Fish Fry, episode 692, 2026. `https://www.eejournal.com/fish_fry/math-first-ai-how-kodamai-embeds-verification-and-trust/`

[26] G. Kreisel and J.-L. Krivine. *Elements of Mathematical Logic: Model Theory*. North-Holland, 1967.

[27] Lean FRO. Lean 4.7.0 release notes (the `omega` tactic). April 2024. `https://lean-lang.org/blog/2024-4-4-lean-470`

[28] S. Libkind and D. I. Spivak. Dynamic task delegation for hierarchical agents. arXiv:2410.08373, 2024.

[29] Anthropic. Introducing the Model Context Protocol. November 2024. `https://www.anthropic.com/news/model-context-protocol`

[30] C. McBride. *Dependently Typed Functional Programs and their Proofs*. Ph.D. thesis, University of Edinburgh, 1999.

[31] C. McBride. Elimination with a motive. In *TYPES 2000*, LNCS 2277, pp. 197–216. Springer, 2002.

[32] C. McBride and J. McKinna. The view from the left. *Journal of Functional Programming* 14(1):69–111, 2004.

[33] P. Morris. *Constructing Universes for Generic Programming*. Ph.D. thesis, University of Nottingham, 2007.

[34] N. Niu and D. I. Spivak. *Polynomial Functors: A Mathematical Theory of Interaction*. Cambridge University Press, 2024 (arXiv:2312.00990).

[35] OpenAI. Introducing Structured Outputs in the API. August 2024. `https://openai.com/index/introducing-structured-outputs-in-the-api/`

[36] M. Presburger. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In *Comptes Rendus du Ier Congrès des Mathématiciens des Pays Slaves*, pp. 92–101, Warsaw, 1929.

[37] R. Prince. An extension of the ellipsis technique. M.Sc. thesis, School of Informatics, University of Edinburgh, 2005.

[38] R. C. S. Prince. *Aspects of the Theory of Containers within Automated Theorem Proving*. Ph.D. thesis, University of Nottingham, 2010.

[39] R. Prince, N. Ghani and C. McBride. Proving properties about lists using containers. In *Functional and Logic Programming (FLOPS 2008)*, LNCS 4989, pp. 97–112. Springer, 2008.

[40] W. Pugh. The Omega test: a fast and practical integer programming algorithm for dependence analysis. In *Supercomputing '91*, pp. 4–13. ACM, 1991.

[41] D. I. Spivak. Poly: an abundant categorical setting for mode-dependent dynamics. arXiv:2005.01894, 2020.

[42] D. I. Spivak. Interactions that reshape the interfaces of the interacting parties. arXiv:2602.17917, 2026.

[43] P. Suter, M. Dotta and V. Kuncak. Decision procedures for algebraic data types with abstractions. In *POPL 2010*, pp. 199–210. ACM, 2010.

[44] T. Zhang, H. B. Sipma and Z. Manna. Decision procedures for recursive data structures with integer constraints. In *IJCAR 2004*, LNCS 3097, pp. 152–167. Springer, 2004.